

Kiến trúc điều
khiển nhúng
real-time và
chuẩn bị bảo vệ

Vai trò của
Embedded Controller
trong ATLC

Từ Single Loop
Firmware đến nhu
cầu RTOS

Kiến thức nền tảng
RTOS cần cho ATLC

Queue-Based
Architecture trong
ATLC

Tổng quan kiến trúc
FreeRTOS Controller

ATLC: Adaptive Traffic Light Controller

Phần 07.1 — Embedded Controller

Motivation & Required Background

RTOS/FreeRTOS vừa đủ để hiểu Phase 15 và bảo vệ kiến trúc controller

Mục tiêu của Section 7.1

ATLC Section 07.1

Kiến trúc điều khiển nhúng real-time và chuẩn bị bảo vệ

Vai trò của Embedded Controller trong ATLC

Từ Single Loop Firmware đến nhu cầu RTOS

Kiến trúc nền tảng RTOS cần cho ATLC

Queue-Based Architecture trong ATLC

Tổng quan kiến trúc FreeRTOS Controller

Vai trò của Embedded Controller trong ATLC

Section này không dạy toàn bộ FreeRTOS. Mục tiêu là giúp nhóm hiểu vì sao Phase 15 là một nâng cấp kiến trúc embedded controller, không chỉ là thêm một thư viện.

- Hiểu vai trò của MCU/embedded controller trong ATLC.
- Nhìn thấy vấn đề của firmware dạng single loop khi trách nhiệm tăng.
- Nắm các concept RTOS/FreeRTOS được dùng thật: task, scheduler, blocked state, queue.
- Hiểu kiến trúc mục tiêu: TaskUARTReceive → rawMessageQueue → TaskPlanParser → planQueue → TaskTrafficFSM.
- Biết cách giải thích Phase 15 khi bảo vệ theo hướng kiến trúc hệ thống.

Kỳ vọng sau phần này

Team không cần thuộc API FreeRTOS, nhưng phải giải thích được controller mới tách trách nhiệm ra sao và vì sao điều đó làm hệ thống dễ bảo trì, dễ debug và ổn định hơn.

(Recap) Vì sao Embedded Controller quan trọng?

ATLC Section 07.1

Kiến trúc điều khiển những real-time và chuẩn bị bảo vệ

Vai trò của Embedded Controller trong ATLC

Từ Single Loop Firmware đến nhu cầu RTOS

Kiến trúc nền tảng RTOS cần cho ATLC

Queue-Based Architecture trong ATLC

Tổng quan kiến trúc FreeRTOS Controller

Vai trò của Embedded Controller trong ATLC

ATLC không chỉ là pipeline AI phát hiện xe. Hệ thống còn cần chuyển khuyến nghị phần mềm thành hành vi điều khiển ổn định ở tầng phần cứng.

- YOLO: phát hiện phương tiện trong video.
- ROI / Counting / Density: biến detection thành dữ liệu giao thông theo hướng.
- Scheduler: đề xuất timing plan ở mức cao.
- Embedded Controller: thực thi timing plan thành chu kỳ đèn cụ thể.

Câu cần nhớ

AI pipeline tạo nhận thức và khuyến nghị. Embedded controller chịu trách nhiệm thực thi điều khiển theo thời gian.

Từ AI Pipeline đến Control Execution

ATLC Section 07.1

Vai trò của Embedded Controller trong ATLC

Kiến trúc điều khiển những real-time và chuẩn bị bảo vệ

Vai trò của Embedded Controller trong ATLC

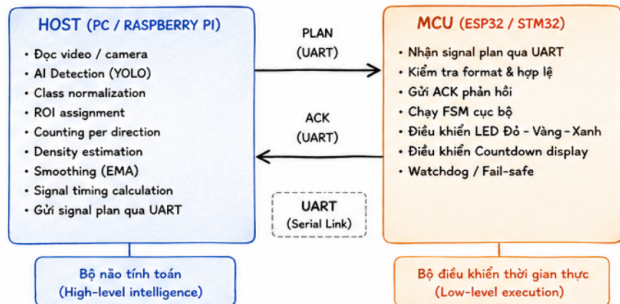
Từ Single Loop Firmware đến nhu cầu RTOS

Kiến trúc nền tảng RTOS cần cho ATLC

Queue-Based Architecture trong ATLC

Tổng quan kiến trúc FreeRTOS Controller

3. PHÂN TẦNG HOST - MCU (2 CỘT)



Hình 1: Phân tầng Host-MCU: host xử lý video và tính plan, MCU thực thi traffic light FSM.

Host và MCU không làm cùng một việc

ATLC Section 07.1

Kiến trúc điều
 khiển nhúng
 real-time và
 chuẩn bị bảo vệ

Vai trò của
 Embedded Controller
 trong ATLC

Từ Single Loop
 Firmware đến nhu
 cầu RTOS

Kiến trúc nền tảng
 RTOS cần cho ATLC

Queue-Based
 Architecture trong
 ATLC

Tổng quan kiến trúc
 FreeRTOS Controller

Vai trò của Embedded Controller trong ATLC

Một lỗi trình bày phổ biến là nói mơ hồ rằng “hệ thống AI điều khiển đèn”. Cách nói đúng hơn là phân tầng rõ ràng.

Host / Raspberry Pi / PC	MCU / ESP32 / Arduino
Đọc camera hoặc video	Nhận plan đã tính sẵn
Chạy YOLO detection	Không chạy YOLO
Tính ROI, count, density	Không xử lý ảnh
Tạo timing plan	Chạy FSM điều khiển đèn
Gửi PLAN message	Xuất LED đỏ/vàng/xanh

Ý nghĩa

Host là tầng tính toán mức cao. MCU là tầng thực thi điều khiển theo thời gian. Hai tầng này cần giao tiếp rõ ràng qua protocol.

Single Loop Firmware là gì?

ATLC Section
07.1

Kiến trúc điều
khiển nhúng
real-time và
chuẩn bị bảo vệ

Vai trò của
Embedded Controller
trong ATLC

Từ Single Loop
Firmware đến nhu
cầu RTOS

Kiến thức nền tảng
RTOS cần cho ATLC

Queue-Based
Architecture trong
ATLC

Tổng quan kiến trúc
FreeRTOS Controller

Từ Single Loop Firmware đến nhu cầu RTOS

Trong firmware Arduino/ESP32 đơn giản, nhiều logic thường được đặt trong một vòng lặp chính.

Ví dụ trong ATLC

Một vòng lặp có thể vừa nhận PLAN từ host, vừa parse message, vừa cập nhật FSM và vừa điều khiển LED.

```
void loop() {  
  
    readSerialInput();  
  
    if (hasNewMessage) {  
        parsePlanMessage();  
        validatePlanTiming();  
        applyPlanImmediately();  
        sendAck();  
    }  
  
    applyFallbackTimingIfNeeded();  
  
    updateTrafficLightFSM();  
    writeTrafficLightPins();  
}
```

Ý chính

Single loop nghĩa là nhiều trách nhiệm được xử lý tuần tự trong cùng một vòng lặp chính.

Phân tích Single Loop theo trách nhiệm

ATLC Section
07.1

Kiến trúc điều
khiển nhúng
real-time và
chuẩn bị bảo vệ

Vai trò của
Embedded Controller
trong ATLC

Từ Single Loop
Firmware đến nhu
cầu RTOS

Kiến trúc nền tảng
RTOS cần cho ATLC

Queue-Based
Architecture trong
ATLC

Tổng quan kiến trúc
FreeRTOS Controller

Từ Single Loop Firmware đến nhu cầu RTOS

Single loop không sai. Nó thường rất phù hợp ở giai đoạn prototype ban đầu. Vấn đề xuất hiện khi số lượng trách nhiệm trong controller tăng lên.

Phần trong loop	Trách nhiệm trong ATLC
<code>readSerialInput()</code>	Nhận message từ host hoặc Raspberry Pi.
<code>parsePlanMessage()</code>	Tách chuỗi <code>PLAN,...</code> thành các trường timing.
<code>validatePlanTiming()</code>	Chặn plan sai format hoặc giá trị không an toàn.
<code>applyPlanImmediately()</code>	Cập nhật plan đang dùng cho FSM.
<code>updateTrafficLightFSM()</code>	Duy trì chu kỳ đèn cục bộ.
<code>sendAck()</code>	Báo host biết message đã được xử lý.

Điểm cần nhận ra

Các phần này thuộc các nhóm trách nhiệm khác nhau: communication, parsing, validation, control execution và protocol

Hạn chế của Single Loop khi hệ thống lớn hơn

ATLC Section
07.1

Kiến trúc điều
khiển nhúng
real-time và
chuẩn bị bảo vệ

Vai trò của
Embedded Controller
trong ATLC

Từ Single Loop
Firmware đến nhu
cầu RTOS

Kiến trúc nền tảng
RTOS cần cho ATLC

Queue-Based
Architecture trong
ATLC

Tổng quan kiến trúc
FreeRTOS Controller

Từ Single Loop Firmware đến nhu cầu RTOS

Single loop dễ hiểu, nhưng bắt đầu hạn chế khi firmware phải xử lý nhiều trách nhiệm real-time.

- Receive, parser, validation và FSM bị trộn gần nhau trong cùng một luồng xử lý.
- Nếu parser xử lý chậm hoặc lỗi, nhịp cập nhật FSM có thể bị ảnh hưởng.
- Khó test riêng từng phần: nhận serial, parse PLAN, validate timing, FSM.
- Polling nhiều làm CPU liên tục kiểm tra dù chưa có message mới.
- Khó mở rộng thêm STATUS message, watchdog, fault recovery hoặc debug task.
- Code dễ trở thành một khối dài, khó đọc và khó bảo trì.

Cách nói cân bằng

Loop-based firmware không sai. Nó chỉ bắt đầu hạn chế khi controller cần nhiều trách nhiệm real-time hơn và cần được tách kiến trúc rõ ràng hơn.

Tư duy chuyển từ Single Loop sang FreeRTOS

ATLC Section
07.1

Kiến trúc điều
khiển nhúng
real-time và
chuẩn bị bảo vệ

Vai trò của
Embedded Controller
trong ATLC

Từ Single Loop
Firmware đến nhu
cầu RTOS

Kiến trúc nền tảng
RTOS cần cho ATLC

Queue-Based
Architecture trong
ATLC

Tổng quan kiến trúc
FreeRTOS Controller

Từ Single Loop Firmware đến nhu cầu RTOS

Phase 15 không chỉ là đổi cú pháp code. Ý chính là tách controller thành các khối xử lý độc lập hơn.

Single loop	FreeRTOS architecture
Một vòng lặp xử lý nhiều việc	Nhiều task, mỗi task có trách nhiệm riêng
Serial và parser nằm gần nhau	Receive task và parser task tách qua queue
Plan hợp lệ có thể sửa timing ngay	Plan hợp lệ thành <code>SignalPlan</code> và đi qua <code>planQueue</code>
FSM chạy chung với logic message	FSM chạy trong task riêng
Khó thêm STATUS/watchdog	Có thể thêm task/timer mới dễ hơn

Thông điệp chính

FreeRTOS giúp chuyển controller từ một vòng lặp đơn khối sang kiến trúc real-time nhiều task, giao tiếp bằng queue.

RTOS là gì trong ngữ cảnh firmware?

ATLC Section 07.1

Kiến trúc điều khiển những real-time và chuẩn bị bảo vệ

Vai trò của Embedded Controller trong ATLC

Từ Single Loop Firmware đến nhu cầu RTOS

Kiến trúc nền tảng RTOS cần cho ATLC

Queue-Based Architecture trong ATLC

Tổng quan kiến trúc FreeRTOS Controller

Kiến trúc nền tảng RTOS cần cho ATLC

Trong firmware những đơn giản, chương trình thường chạy theo một vòng lặp chính. Khi hệ thống lớn hơn, firmware phải xử lý nhiều việc cùng lúc hơn.

- Nhận message từ host.
- Parse và validate PLAN.
- Duy trì chu kỳ đèn.
- Gửi ACK/NACK.
- Chuẩn bị STATUS hoặc watchdog.
- Ghi log/debug khi cần.

RTOS là gì?

RTOS là lớp kernel nhỏ giúp firmware tổ chức nhiều luồng xử lý logic thành các task, cho phép task chờ sự kiện, giao tiếp với nhau và chia sẻ CPU có kiểm soát.

FreeRTOS trong ngữ cảnh ATLC

ATLC Section 07.1

Kiến trúc điều
 khiển những
 real-time và
 chuẩn bị bảo vệ

Vai trò của
 Embedded Controller
 trong ATLC

Từ Single Loop
 Firmware đến nhu
 cầu RTOS

Kiến trúc nền tảng
 RTOS cần cho ATLC

Queue-Based
 Architecture trong
 ATLC

Tổng quan kiến trúc
 FreeRTOS Controller

Kiến trúc nền tảng RTOS cần cho ATLC

FreeRTOS không được dùng để làm firmware “phức tạp cho chuyên nghiệp”. Nó được dùng vì controller cần nhiều trách nhiệm tách biệt.

- Tách nhận serial khỏi parse/validate.
- Tách parse/validate khỏi traffic FSM.
- Cho parser chờ message thay vì polling liên tục.
- Cho các task trao đổi dữ liệu qua queue.
- Tạo nền cho ACK/NACK, STATUS, watchdog và fault recovery.

Cách nói khi bảo vệ

FreeRTOS enabled a transition from a monolithic controller implementation toward a modular real-time embedded architecture.

Các concept FreeRTOS cần hiểu trong ATLC

ATLC Section 07.1

Kiến trúc điều
khiển nhúng
real-time và
chuẩn bị bảo vệ

Vai trò của
Embedded Controller
trong ATLC

Từ Single Loop
Firmware đến nhu
cầu RTOS

Kiến trúc nền tảng
RTOS cần cho ATLC

Queue-Based
Architecture trong
ATLC

Tổng quan kiến trúc
FreeRTOS Controller

Kiến trúc nền tảng RTOS cần cho ATLC

Team không cần học toàn bộ FreeRTOS để bảo vệ ATLC. Nhưng cần hiểu các concept thật sự xuất hiện trong kiến trúc controller.

Concept	Cách hiểu trong ATLC
Task	Một luồng xử lý riêng: nhận serial, parse plan, hoặc chạy FSM.
Scheduler	Cơ chế quyết định task nào được chạy khi có nhiều task cùng tồn tại.
Blocked state	Task chờ message/event mà không polling liên tục.
Queue	Kênh truyền dữ liệu giữa task theo producer-consumer.
FSM task	Task duy trì chu kỳ đèn cục bộ, không phụ thuộc từng frame YOLO.
ACK/NACK	Protocol response để host biết PLAN được chấp nhận hay bị từ chối.
Safe boundary	Thời điểm an toàn để áp dụng plan mới mà không restart FSM tùy tiện.

Task là gì?

ATLC Section 07.1

Kiến trúc điều khiển những real-time và chuẩn bị bảo vệ

Vai trò của Embedded Controller trong ATLC

Từ Single Loop Firmware đến nhu cầu RTOS

Kiến trúc nền tảng RTOS cần cho ATLC

Queue-Based Architecture trong ATLC

Tổng quan kiến trúc FreeRTOS Controller

Kiến trúc nền tảng RTOS cần cho ATLC

Một task là một luồng xử lý logic trong firmware. Thông thường mỗi task chạy một vòng lặp riêng và có trách nhiệm cụ thể.

```
void TaskUARTReceive(void *pv) {
    for (;;) {
        readSerialLine();
        sendRawMessageToQueue();
    }
}

void TaskPlanParser(void *pv) {
    for (;;) {
        waitRawMessage();
        parseAndValidatePlan();
    }
}
```

- Task không phải là một hàm chạy một lần.
- Task thường sống trong suốt runtime.
- Mỗi task nên có một nhiệm vụ chính.
- Task không nên biết quá nhiều chi tiết nội bộ của task khác.

Mapping sang ATLC

TaskUARTReceive nhận message. TaskPlanParser parse/validate. TaskTrafficFSM duy trì chu kỳ đèn.

Scheduler trong RTOS làm gì?

ATLC Section 07.1

Kiến trúc điều
khiển nhúng
real-time và
chuẩn bị bảo vệ

Vai trò của
Embedded Controller
trong ATLC

Từ Single Loop
Firmware đến nhu
cầu RTOS

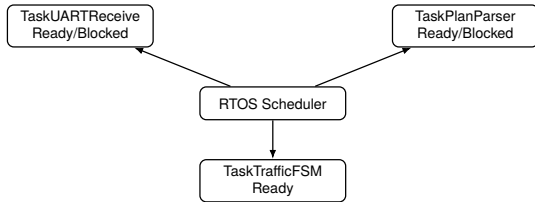
Kiến thức nền tảng
RTOS cần cho ATLC

Queue-Based
Architecture trong
ATLC

Tổng quan kiến trúc
FreeRTOS Controller

Kiến thức nền tảng RTOS cần cho ATLC

Khi có nhiều task, MCU vẫn chỉ có tài nguyên CPU hữu hạn.
Scheduler quyết định task nào được chạy tại một thời điểm.



- Task có việc để làm sẽ ở trạng thái ready.
- Task chưa có dữ liệu có thể chờ ở trạng thái blocked.
- Scheduler chọn task phù hợp để chạy.

Blocked State: chờ message mà không polling

ATLC Section 07.1

Kiến trúc điều khiển những real-time và chuẩn bị bảo vệ

Vai trò của Embedded Controller trong ATLC

Từ Single Loop Firmware đến nhu cầu RTOS

Kiến trúc nền tảng RTOS cần cho ATLC

Queue-Based Architecture trong ATLC

Tổng quan kiến trúc FreeRTOS Controller

Kiến trúc nền tảng RTOS cần cho ATLC

Một task có thể tạm dừng để chờ sự kiện. Trong thời gian đó, task không lãng phí CPU để kiểm tra liên tục.

Polling-heavy	Event-driven / blocked
Liên tục hỏi: có message chưa?	Chờ trên queue đến khi có message
CPU kiểm tra lặp lại	CPU có thể chạy task khác
Code dễ có nhiều kiểm tra thủ công	Logic rõ hơn: có dữ liệu thì xử lý
Dễ lẫn với FSM timing	Parser và FSM tách nhịp xử lý

Ví dụ ATLC

`TaskPlanParser` có thể block để chờ `rawMessageQueue`.

Nhưng `TaskTrafficFSM` không nên block forever vì còn phải cập nhật trạng thái đèn và countdown.

Queue là gì?

ATLC Section
07.1

Kiến trúc điều
khiển nhúng
real-time và
chuẩn bị bảo vệ

Vai trò của
Embedded Controller
trong ATLC

Từ Single Loop
Firmware đến nhu
cầu RTOS

Kiến trúc nền tảng
RTOS cần cho ATLC

Queue-Based
Architecture trong
ATLC

Tổng quan kiến trúc
FreeRTOS Controller

Queue-Based Architecture trong ATLC

Queue là kênh truyền dữ liệu giữa các task.

- Một task tạo dữ liệu và gửi vào queue.
- Task khác lấy dữ liệu ra để xử lý.
- Sender không cần gọi trực tiếp receiver.
- Receiver không cần biết sender đang xử lý nội bộ như thế nào.
- Cách này tạo kiến trúc producer–consumer.

Ví dụ đời thường

Task gửi giống như người bỏ thư vào hộp thư. Task nhận giống như người lấy thư ra xử lý. Hai người không cần gặp trực tiếp nhau.

Trong ATLC

`rawMessageQueue` truyền message thô. `planQueue` truyền signal plan đã parse và validate.

Queue tạo mô hình Producer–Consumer

ATLC Section 07.1

Kiến trúc điều
 khiển nhúng
 real-time và
 chuẩn bị bảo vệ

Vai trò của
 Embedded Controller
 trong ATLC

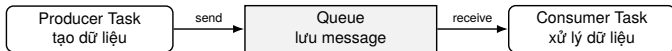
Từ Single Loop
 Firmware đến nhu
 cầu RTOS

Kiến trúc nền tảng
 RTOS cần cho ATLC

Queue-Based
 Architecture trong
 ATLC

Tổng quan kiến trúc
 FreeRTOS Controller

Queue-Based Architecture trong ATLC



Sender và receiver không gọi trực tiếp nhau.
Queue là ranh giới giao tiếp giữa hai task.

Ý nghĩa kiến trúc

Nếu thay đổi producer, consumer không nhất thiết phải sửa. Nếu thay đổi consumer, producer cũng không cần biết chi tiết.

rawMessageQueue: truyền message thô

ATLC Section
07.1

Kiến trúc điều
khiển nhúng
real-time và
chuẩn bị bảo vệ

Vai trò của
Embedded Controller
trong ATLC

Từ Single Loop
Firmware đến nhu
cầu RTOS

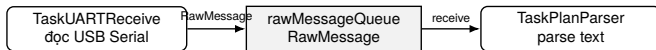
Kiến trúc nền tảng
RTOS cần cho ATLC

Queue-Based
Architecture trong
ATLC

Tổng quan kiến trúc
FreeRTOS Controller

Queue-Based Architecture trong ATLC

rawMessageQueue nằm giữa task nhận serial và task parser.



```
struct RawMessage {  
    char text[64];    // "PLAN, 17, 25, 15, 3, 1"  
};
```

Điểm cần hiểu

Task nhận serial chỉ cần tạo `RawMessage`. Nó chưa cần hiểu PLAN hợp lệ hay không. Việc hiểu nội dung là trách nhiệm của parser.

planQueue: chỉ truyền plan đã validate

ATLC Section 07.1

Kiến trúc điều
 khiển nhúng
 real-time và
 chuẩn bị bảo vệ

Vai trò của
 Embedded Controller
 trong ATLC

Từ Single Loop
 Firmware đến nhu
 cầu RTOS

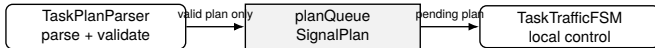
Kiến trúc nền tảng
 RTOS cần cho ATLC

Queue-Based
 Architecture trong
 ATLC

Tổng quan kiến trúc
 FreeRTOS Controller

Queue-Based Architecture trong ATLC

Sau khi parser kiểm tra message, dữ liệu thô được chuyển thành object điều khiển rõ ràng hơn.



```
struct SignalPlan {  
    int plan_id;  
    int green_a;  
    int green_b;  
    int yellow;  
    int all_red;  
};
```

Điểm cần hiểu sâu

Không phải mọi text từ host đều được đưa vào FSM. Chỉ message parse đúng và validate đúng mới trở thành `SignalPlan`.

Vì sao dùng Queue thay vì gọi hàm trực tiếp?

ATLC Section
07.1

Kiến trúc điều
khiển nhúng
real-time và
chuẩn bị bảo vệ

Vai trò của
Embedded Controller
trong ATLC

Từ Single Loop
Firmware đến nhu
cầu RTOS

Kiến trúc nền tảng
RTOS cần cho ATLC

Queue-Based
Architecture trong
ATLC

Tổng quan kiến trúc
FreeRTOS Controller

Queue-Based Architecture trong ATLC

Nếu parser gọi trực tiếp FSM, hai phần sẽ phụ thuộc chặt vào nhau hơn. Queue tạo một ranh giới rõ giữa parsing và control execution.

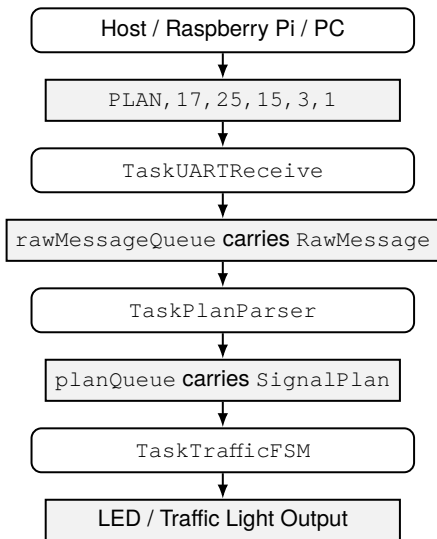
Gọi trực tiếp	Gửi qua queue
Parser biết nhiều về FSM	Parser chỉ tạo <code>SignalPlan</code>
Plan có thể được áp dụng ngay	FSM quyết định khi nào áp dụng
Khó test parser riêng	Có thể test parser bằng queue/log
Khó thay receive layer	Có thể thay fake producer bằng UART task
Dễ restart FSM mỗi message	Có thể dùng pending plan và safe boundary

Ý nghĩa trong ATLC

Queue giúp tách communication, parsing và FSM. Đây là điểm làm Phase 15 trở thành nâng cấp kiến trúc.

Task Pipeline ở mức kiến trúc

Tổng quan kiến trúc FreeRTOS Controller



ATLC Section
07.1

Kiến trúc điều
 khiển những
 real-time và
 chuẩn bị bảo vệ

Vai trò của
 Embedded Controller
 trong ATLC

Từ Single Loop
 Firmware đến nhu
 cầu RTOS

Kiến trúc nền tảng
 RTOS cần cho ATLC

Queue-Based
 Architecture trong
 ATLC

Tổng quan kiến trúc
 FreeRTOS Controller

TaskUARTReceive: Serial Boundary Task

ATLC Section 07.1

Kiến trúc điều
 khiển nhúng
 real-time và
 chuẩn bị bảo vệ

Vai trò của
 Embedded Controller
 trong ATLC

Từ Single Loop
 Firmware đến nhu
 cầu RTOS

Kiến trúc nền tảng
 RTOS cần cho ATLC

Queue-Based
 Architecture trong
 ATLC

Tổng quan kiến trúc
 FreeRTOS Controller

Tổng quan kiến trúc FreeRTOS Controller

`TaskUARTReceive` là ranh giới giữa host và firmware.

- Đọc dữ liệu từ USB Serial hoặc UART-like serial port.
- Gom message theo dòng, ví dụ kết thúc bằng newline.
- Đóng gói message thành `RawMessage`.
- Gửi `RawMessage` vào `rawMessageQueue`.

Nguyên tắc tách trách nhiệm

Task này không tính green time, không điều khiển LED và không tự chạy FSM. Nó chỉ chịu trách nhiệm nhận dữ liệu thô từ communication boundary.

TaskPlanParser: Parsing và Validation

ATLC Section 07.1

Kiến trúc điều
khiển những
real-time và
chuẩn bị bảo vệ

Vai trò của
Embedded Controller
trong ATLC

Từ Single Loop
Firmware đến nhu
cầu RTOS

Kiến trúc nền tảng
RTOS cần cho ATLC

Queue-Based
Architecture trong
ATLC

Tổng quan kiến trúc
FreeRTOS Controller

Tổng quan kiến trúc FreeRTOS Controller

Parser task chuyển message thô thành plan có cấu trúc.

- Nhận RawMessage từ rawMessageQueue.
- Parse format
PLAN, <id>, <greenA>, <greenB>, <yellow>, <allRed>.
- Kiểm tra số field và kiểu dữ liệu.
- Validate giới hạn timing.
- Nếu hợp lệ, tạo SignalPlan và gửi vào planQueue.

Điểm bảo vệ

MCU không nên tin mọi message từ host. Plan phải được parse và validate trước khi đưa vào FSM.

TaskTrafficFSM: Local Autonomous Controller

ATLC Section 07.1

Kiến trúc điều
 khiển những
 real-time và
 chuẩn bị bảo vệ

Vai trò của
 Embedded Controller
 trong ATLC

Từ Single Loop
 Firmware đến nhu
 cầu RTOS

Kiến trúc nền tảng
 RTOS cần cho ATLC

Queue-Based
 Architecture trong
 ATLC

Tổng quan kiến trúc
 FreeRTOS Controller

Tổng quan kiến trúc FreeRTOS Controller

FSM task chịu trách nhiệm thực thi chu kỳ đèn.

- Nhận `SignalPlan` hợp lệ từ `planQueue`.
- Duy trì trạng thái `GREEN_A`, `YELLOW_A`, `ALL_RED`, `GREEN_B`,
...
- Điều khiển LED đỏ, vàng, xanh.
- Cập nhật countdown hoặc state log nếu cần.
- Tiếp tục chạy ngay cả khi host chưa gửi plan mới.

Điểm quan trọng

Host gửi high-level plan. ESP32 thực thi local FSM. AI inference và control execution được tách ra.

Local FSM không phụ thuộc từng frame YOLO

ATLC Section 07.1

Kiến trúc điều khiển những real-time và chuẩn bị bảo vệ

Vai trò của Embedded Controller trong ATLC

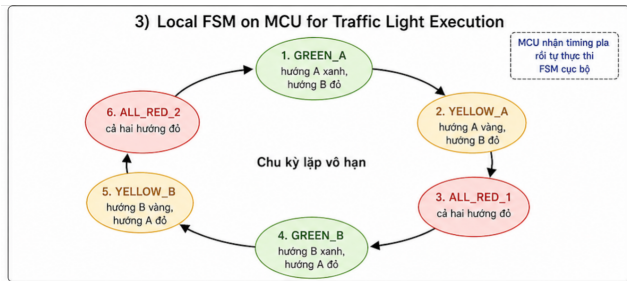
Từ Single Loop Firmware đến nhu cầu RTOS

Kiến trúc nền tảng RTOS cần cho ATLC

Queue-Based Architecture trong ATLC

Tổng quan kiến trúc FreeRTOS Controller

Tổng quan kiến trúc FreeRTOS Controller



Hình 2: ESP32 thực thi traffic light FSM cục bộ dựa trên timing plan nhận từ host.

Ý nghĩa

Host FPS có thể dao động, nhưng MCU vẫn duy trì chu kỳ đèn dựa trên plan hiện tại. Đây là điểm tách AI inference khỏi control execution.

PLAN, ACK và NACK ở mức khái niệm

ATLC Section 07.1

Kiến trúc điều
khiển nhúng
real-time và
chuẩn bị bảo vệ

Vai trò của
Embedded Controller
trong ATLC

Từ Single Loop
Firmware đến nhu
cầu RTOS

Kiến trúc nền tảng
RTOS cần cho ATLC

Queue-Based
Architecture trong
ATLC

Tổng quan kiến trúc
FreeRTOS Controller

Tổng quan kiến trúc FreeRTOS Controller

- Host gửi: PLAN, 17, 25, 15, 3, 1
- Nếu plan hợp lệ: ACK, 17
- Nếu format hoặc timing sai: NACK, 17, INVALID_GREEN

Cách hiểu đúng

ACK xác nhận message/plan được nhận và chấp nhận ở mức protocol. ACK không chứng minh plan tối ưu hoặc hệ thống đủ an toàn ngoài thực tế.

Safe Plan Apply

ATLC Section 07.1

Kiến trúc điều
khiển nhúng
real-time và
chuẩn bị bảo vệ

Vai trò của
Embedded Controller
trong ATLC

Từ Single Loop
Firmware đến nhu
cầu RTOS

Kiến trúc nền tảng
RTOS cần cho ATLC

Queue-Based
Architecture trong
ATLC

Tổng quan kiến trúc
FreeRTOS Controller

Tổng quan kiến trúc FreeRTOS Controller

Một plan mới không nên làm FSM restart ngay giữa chu kỳ.

- Plan mới có thể được lưu như pending plan.
- FSM quyết định thời điểm áp dụng an toàn hơn, ví dụ ở cycle boundary.
- Tránh hiện tượng đèn đổi pha quá đột ngột theo từng frame.
- Giúp behavior dễ giải thích hơn khi xem log.

Ý nghĩa

Adaptive không có nghĩa là đổi đèn ngay lập tức ở mọi frame.
Adaptive cần đi kèm execution stability.

Watchdog, STATUS và Future Scalability

ATLC Section 07.1

Kiến trúc điều
khiển nhúng
real-time và
chuẩn bị bảo vệ

Vai trò của
Embedded Controller
trong ATLC

Từ Single Loop
Firmware đến nhu
cầu RTOS

Kiến trúc nền tảng
RTOS cần cho ATLC

Queue-Based
Architecture trong
ATLC

Tổng quan kiến trúc
FreeRTOS Controller

Tổng quan kiến trúc FreeRTOS Controller

Kiến trúc task/queue tạo nền để mở rộng mà không phải viết lại toàn bộ firmware.

- STATUS: MCU báo state, remaining time và health về host.
- Watchdog: phát hiện host timeout và chuyển fallback timing.
- Software timer: gửi status định kỳ.
- Runtime debug: stack high-water mark, heap check, queue creation failure.
- Raspberry Pi deployment: host edge device gửi plan xuống MCU thực tế hơn.

Điểm bảo vệ

Phase 15 tạo nền kiến trúc để thêm reliability features mà không phải thiết kế lại toàn bộ controller.

Tổng kết kiến thức nền tảng cần nắm

ATLC Section
07.1

Kiến trúc điều
 khiển nhúng
 real-time và
 chuẩn bị bảo vệ

Vai trò của
 Embedded Controller
 trong ATLC

Từ Single Loop
 Firmware đến nhu
 cầu RTOS

Kiến trúc nền tảng
 RTOS cần cho ATLC

Queue-Based
 Architecture trong
 ATLC

Tổng quan kiến trúc
 FreeRTOS Controller

Tổng quan kiến trúc FreeRTOS Controller

Sau phần nền tảng, team cần trả lời được 5 câu hỏi sau:

- Single loop firmware là gì và vì sao nó phù hợp ở prototype ban đầu?
- Vì sao single loop bắt đầu hạn chế khi receive, parser, FSM, ACK, STATUS và watchdog cùng tăng độ phức tạp?
- Task trong FreeRTOS giúp tách trách nhiệm như thế nào?
- Queue giúp TaskUARTReceive, TaskPlanParser và TaskTrafficFSM giao tiếp ra sao?
- Vì sao ESP32 nên chạy FSM cục bộ thay vì host điều khiển từng LED theo từng frame?

Câu chốt khi bảo vệ

Phase 15 không chỉ thêm FreeRTOS. Phase 15 tái cấu trúc embedded controller từ loop-based firmware sang kiến trúc real-time nhiều task, giao tiếp qua queue và thực thi FSM cục bộ.

Chuyển tiếp sang Section 7.2

ATLC Section 07.1

Kiến trúc điều
khiển nhúng
real-time và
chuẩn bị bảo vệ

Vai trò của
Embedded Controller
trong ATLC

Từ Single Loop
Firmware đến nhu
cầu RTOS

Kiến trúc nền tảng
RTOS cần cho ATLC

Queue-Based
Architecture trong
ATLC

Tổng quan kiến trúc
FreeRTOS Controller

Tổng quan kiến trúc FreeRTOS Controller

Section 7.1 cung cấp động lực và kiến thức nền. Section 7.2 sẽ đi sâu vào triển khai cụ thể.

- Cấu trúc file firmware Phase 15.
- Cách tạo queue và task.
- Cách parse PLAN message.
- Cách FSM nhận pending plan và cập nhật LED.
- Cách debug ACK/NACK, queue và timing.